

Contents

1	Artifact README	1
1.1	Artifact Contents	1
1.2	System Requirements	2
1.2.1	Help & References	2
1.3	Credential Setup	2
1.3.1	OpenAI GPT Setup	2
1.3.2	AWS Bedrock Setup	3
1.4	Smoke Test: Kick-the-Tires Instructions	4
1.4.1	Load the Docker image	4
1.4.2	Exposing Ollama as a Service	4
1.4.3	Prepare the <code>.env</code> file	5
1.4.4	Setting the model details	5
1.4.5	Start the container	5
1.4.6	Flags supported by tool (NLUnsat)	6
1.4.7	Run the smoke test inside the container	7
1.4.8	Check the output on the host	7
1.5	Reproducibility Runs: Complete Evaluation Instructions	8
1.5.1	Model list for evaluation	8
1.5.2	Directory Naming Convention	8
1.5.3	Additional notes on folder structure to save logs	8
1.5.4	CLI flags supported by the tool	9
1.5.5	Running all 193 benchmarks (baseline)	9
1.5.6	For all 193 benchmarks with No Gamma experiment (NoGamma Ablation)	10
1.5.7	For all 193 benchmarks with No Lemma experiment (NoLemma Ablation)	10
1.6	Reusability Notes for benchmarks	10
1.7	Troubleshooting	11

1 Artifact README

This README is the top-level documentation for the artifact submission for **NLUNSAT** tool.

1.1 Artifact Contents

The Zenodo records contains the following:

- **Instructions.md** to run the scripts in **scripts.zip** file to generate the results for the tables and claims in the paper against pre-existing experiment logs.
- **scripts.zip** contains scripts to generate the table datas, chart images and PDFs of the plots that have been used in the paper.
- **toolimage.tar.gz** is a docker compatible tarball zip of the tool.
- **logFiles.zip** contains the experimental logs used to generate the plots and timing results as presented in the paper.
- **README.md** is the top-level file describing the instructions, requirements and details about how to run the artifact (tool presented in **toolimage.tar.gz** docker image).
- **UnderstandingLogFiles.md** explains how to interpret the log file generated when running our tool against a given model and benchmark.

The Docker image already contains the code, dependencies, benchmarks, and scripts needed to run the artifact.

1.2 System Requirements

The artifact is intended to be exercised through Docker on a Linux-compatible `x86_64` host. We have tested it on Ubuntu and Amazon Machine images supporting `x86_64`. Running from inside the docker increases time by 1.5x.

Minimum requirements:

1. Docker Engine or Docker Desktop with the `docker` CLI available.
2. Internet access.
3. One of the following model-access configurations:
 - OpenAI API access for GPT-based runs.
 - AWS credentials with Amazon Bedrock model access.
 - Local Ollama server serving model for inference (If local LLM runs are required).
4. A working python version above 3.12+. Versions below 3.10 may not work.
5. Running Ollama locally and expose it to run local LLM models.
6. Must have Ubuntu/Debian machine with 16GB primary memory (RAM) with 8 cores (CPU). We tested on this minimum machine configuration.

1.2.1 Help & References

- [Getting started with Docker](#)
- [Install Docker Engine](#)
- [Starting with Ollama](#)
- [Serving Ollama via docker or localhost](#)
- [Ollama FAQs](#)
- [Running Ollama Docker](#)
- [Installing Python on Ubuntu Machines](#)
- [Download Python](#)

Notes:

1. The default `runUnsat.sh` workflow calls `parallel.sh`, and `parallel.sh` is currently configured to use Amazon Bedrock with model `openai.gpt-oss-120b-1:0` by default.
2. If you want to run the artifact with OpenAI instead of Bedrock, run `main.py` manually with `--usegpt` as shown below, or edit `parallel.sh` via nano (presented in Docker).
3. The Docker image contains the software stack already installed. Reviewers do not need install extra dependencies.

We have tested and check for the following LLM models.

Model / Configuration	Directory Name	Model Name
GPT-OSS-20B (Local LLM)	<code>gpt20b</code>	<code>gpt-oss:20b</code>
GPT-OSS-120B (Amazon Bedrock)	<code>gpt120b</code>	<code>openai.gpt-oss-120b-1:0</code>
Claude (Amazon Bedrock)	<code>claude</code>	<code>us.anthropic.claude-opus-4-6-v1</code>
Qwen-30B (Amazon Bedrock)	<code>qwen30b</code>	<code>qwen.qwen3-coder-30b-a3b-v1:0</code>
Maverick (Amazon Bedrock)	<code>maverick</code>	<code>us.meta.llama4-maverick-17b-instruct-v1:0</code>

1.3 Credential Setup

1.3.1 OpenAI GPT Setup

This project uses the environment variable `CHAT_OPENAI_API_KEY` for OpenAI-backed runs.

How to get an OpenAI API key:

1. Sign in to the OpenAI API platform.

2. Open the API key page: <https://platform.openai.com/api-keys>
3. Create a new secret key for the project you want to bill.
4. Copy the key immediately and place it in `.env` as `CHAT_OPENAI_API_KEY=...`

Example:

```
CHAT_OPENAI_API_KEY=sk-proj-...
```

Relevant OpenAI documentation:

- [API key page](#)
- [Help article on finding API keys](#)
- [API key safety](#)
- [Current model guidance](#)

OpenAI run example from inside the container:

```
uv run main.py \
-i benchmarks/gt_new/gcd01-1.smt2_5.json \
-t 1 -v \
--usegpt \
--model gpt-5.1 \
--stop
```

`--stop` performs a short sanity check and exits after the initial lemma-generation stage.

1.3.2 AWS Bedrock Setup

The default smoke test uses Amazon Bedrock through `boto3`.

How to prepare AWS access:

1. Use an AWS account that can invoke Bedrock models.
2. Ensure your IAM identity has permission to use Amazon Bedrock and the target model.
3. Ensure the account can access the required model in the chosen AWS region.
4. Provide credentials to the container through `.env`.

The simplest `.env` layout is:

```
AWS_REGION=us-east-1
AWS_BEARER_TOKEN_BEDROCK=ABSQ.....
```

If you use temporary credentials from AWS STS or AWS IAM Identity Center, set `AWS_SESSION_TOKEN` as well.

How to get AWS credentials:

1. If you already use the AWS CLI locally, you may already have credentials for the same account.
2. For long-lived IAM user credentials, an IAM administrator can create an access key in the AWS console.
3. For temporary credentials, use your organization's normal AWS login or SSO flow and copy the resulting environment variables into `.env`.

Relevant AWS documentation:

- [Boto3 credentials guide](#)
- [Amazon Bedrock model access](#)
- [IAM access key management](#)
- [AWS CLI installation](#)

Bedrock run example from inside the container:

```
uv run main.py \
-i benchmarks/gt_new/gcd01-1.smt2_5.json \
-t 1 -v \
--usebedrock \
--model openai.gpt-oss-120b-1:0 \
--stop
```

If Bedrock returns an access error on first use, verify model access and wait a few minutes for any initial subscription step to complete.

1.4 Smoke Test: Kick-the-Tires Instructions

Run the following commands from the directory where you downloaded the artifacts.

1.4.1 Load the Docker image

```
# Load the docker image
docker load --input toolimage.tar.gz

# Check if the image loaded properly
docker image ls nlunsat-tool
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
nlunsat-tool:latest	3a88dfc1b460	3.01GB	574MB	U

The loaded image should appear as `nlunsat-tool:latest`.

Note: `Vim` and `Neovim` packages have been installed along with `nano`. You need to edit some script files as instructed for model changes.

1.4.2 Exposing Ollama as a Service

Ollama can be accessed over the network interface so that other machines in the same network can make calls to it over `11434` port.

Details: <https://aident.ai/blog/how-to-expose-ollama-service-api-to-network>

```
$ sudo systemctl edit ollama.service

# Add this
# Change the server IP.
[Service]
Environment="OLLAMA_HOST=0.0.0.0"
Environment="OLLAMA_ORIGINS=http://172.27.21.160:5173"

$ sudo systemctl daemon-reload
$ sudo systemctl restart ollama

# Check if the serve is happening.
$ sudo netstat -tuln | grep 11434
tcp        0      0 127.0.0.1:11434      0.0.0.0:*           LISTEN
```

The Ollama base URL in this case will be the following

Ollama Base URL: `172.27.21.160:11434`

Changing the `172.27.21.160` to `localhost` or `0.0.0.0` or `127.0.0.1` will only allow local services to access Ollama LLMs.

1.4.3 Prepare the `.env` file

- Create a file named `.env` in the same directory from which you will run `docker run`.
- The container receives these values through `--env-file .env`.
- The default smoke test uses Amazon Bedrock.
- An example `.env` file is shown below. The key values shown here are dummy.
- If you intend to use `Ollama` to run local models, change `BASE_URL`, `BASE_URL_156` and `BASE_URL_160` to `http://<host>:<port>` where Ollama server is serving the model.
- Ideally it must be `http://localhost:11434` if running local LLM models on host machine where docker image is run.

```
BASE_URL=http://172.27.21.156:11434
BASE_URL_156=http://172.27.21.156:11434
BASE_URL_160=http://172.27.21.160:11434
CHAT_OPENAI_API_KEY=sk-proj-XXXXXXXXXXXXXXXXXXXXXXXXXXXX
AWS_BEARER_TOKEN_BEDROCK=ABSKQXXXXXXXXXXXXXXXXXXXXXXXXX
AWS_REGION=us-east-1
```

Here we used two Ollama instances one running on machine `http://172.27.21.156:11434` and other on `http://172.27.21.160:11434` all serving on with port 11434.

Important:

1. `AWS_BEARER_TOKEN_BEDROCK` is checked by the current code and must be non-empty to run AWS Bedrock models via `--usebedrock` flag.
2. If you want to run an OpenAI GPT model instead, you need to set `CHAT_OPENAI_API_KEY` with actual key values:

```
CHAT_OPENAI_API_KEY=sk-proj-XXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

You can keep both the OpenAI and AWS entries in the same `.env` file.

1.4.4 Setting the model details

Please change the respective `parallel*.sh` files to set the model name to run the experiment with. `BASE_URL_156` ollama instance is used with `--use156` and `BASE_URL_160` ollama instance is used with `--use160` flag. Default model setting is `--usebedrock --model openai.gpt-oss-120b-1:0` via AWS Bedrock gateway.

```
# Running with AWS Bedrock models
--usebedrock --model <model_name>

# Running with OpenAI API (ChatGPT)
--usegpt --model <model_name>

# Running with local OLLAMA
--use156 --model <model_name>
--use160 --model <model_name>
```

Model names and configurations have been kept commented for easier use in the scripts.

1.4.5 Start the container

This mount the current working directory as supplied by `$PWD` into a folder `/export` inside the docker. It can be used to interact with host filesystem for copy or move operations.

A docker container of the name `nlunsat` is created. It can be used everytime to run the tool. If the `docker` container feels sluggish (slow and unresponsive) use a higher CPU and memory limit. Running from inside the docker increases time by 1.5x.

- [Configuring Docker Limits](#)

```
# Need to be run only once.
docker run -v $PWD:/export --name nlunsat \
  --env-file .env -dit nlunsat-tool:latest

# Need to be run only once. Running with higher CPU and memory.
docker run -v $PWD:/export --name nlunsat \
  --memory="12g" --cpus="6" \
  --env-file .env -dit nlunsat-tool:latest

# Needs to be run everytime tool needs to run.
docker start -ai nlunsat
```

The container starts in `/app` folder from inside the container, exposing a bash shell to run commands.

1.4.6 Flags supported by tool (NLUnsat)

```
Bytecode compiled 8900 files in 470ms
usage: main.py [-h] -i INPUTFILE [-bench BENCHMARK]
               [-t ITERATIONS] [-v] [-dbg] [-u1] [-u2]
               [-C] [-B] [-W] -M MODEL
               [-V FUZZTIME] [-Z Z3TIME] [-S SHAREDLIB]
               [-a MINLEMMA] [-b MAXLEMMA] [-g ADDGAMMA] [-l ADDLEMMA]
```

A script that accepts arguments.

options:

<code>-h, --help</code>	show this help message and exit
<code>-i, --inputFile INPUTFILE</code>	relative path to json file containing information about the benchmarks
<code>-bench, --benchName BENCHMARK</code>	Name of the benchmark file.
<code>-t, --iterations ITERATIONS</code>	Number of iterations made to LLM.
<code>-v, --verbose</code>	Enable verbose output.
<code>-dbg, --debug</code>	Enable Debug wrapped logs or information.
<code>-u1, --use156</code>	Use the 156 local server.
<code>-u2, --use160</code>	Use the 160 local server.
<code>-C, --usegpt</code>	Use ChatGPT models from OPENAI.
<code>-B, --usebedrock</code>	Set the .env file with <code>CHAT_OPENAI_API_KEY</code> Use LLM models from AWS Bedrock.
	Set the .env file with <code>AWS_BEARER_TOKEN_BEDROCK</code> and <code>AWS_REGION</code>
<code>-W, --stop</code>	Stop running the framework.
<code>-M, --model MODEL</code>	Specify model name to use.
<code>-V, --fuzztime FUZZTIME</code>	Specify Fuzzer timeout in seconds. (Default 20s)
<code>-Z, --z3time Z3TIME</code>	Specify Z3 timeout in seconds. (Default 20s)
<code>-S, --sharedLib SHAREDLIB</code>	

```

                                Link to the shared library. (Default "None")
-a, --minLemma MINLEMMA
                                Minimum number of lemmas to ask
                                LLM for at a time. (Default 1)
-b, --maxLemma MAXLEMMA
                                Maximum number of lemmas to ask
                                LLM for at a time (Default 5)
-g, --addGamma ADDGAMMA
                                If set zero then does not add input
                                output of functions to the z3 solver
-l, --addLemma ADDLEMMA
                                If set zero then does not add
                                LLM generated lemmas to the z3 solver

```

1.4.7 Run the smoke test inside the container

You can change the model in `parallel.sh` file if required, by manually editing it. Some model configurations have been presented below, use any one of them for the test. `BASE_URL_156` ollama instance is used with `--use156` and `BASE_URL_160` ollama instance is used with `--use160` flag. Change in the `parallel.sh` files accordingly.

```

model="--use156 --model gpt-oss:20b"
model="--use160 --model gpt-oss:20b"
model="--usebedrock --model us.meta.llama4-maverick-17b-instruct-v1:0"
model="--usebedrock --model qwen.qwen3-coder-30b-a3b-v1:0"
model="--usebedrock --model us.anthropic.claude-opus-4-6-v1"
model="--usebedrock --model openai.gpt-oss-120b-1:0"
model="--usegpt --model gpt-5.1"

```

```

# Check the help for more details.
uv run main.py --help

```

```

# Run the smoke test.
./runUnsat.sh benchSmokeTest
mv logFilesAll/* /export/

```

This runs the smoke test and moves the log files from inside the container into the folder `$PWD` as mounted while running the docker. Use the [UnderstandingLogFiles.md](#) to understand the log files better.

1.4.8 Check the output on the host

After the move completes, the host directory should contain four log files copied from the container. Their names are prefixed by the benchmark order, for example:

```

1_gcd01-1.smt2_5.json
2_recHanoi03-2.smt2_6.json
3_id_b2_o3.smt2.1_0.json
4_bench_010_clz_popcount.smt2_3.json

```

During execution, the container prints progress lines such as:

```

benchmark: gcd01-1.smt2_5.json
...
gcd01-1.smt2_5.json Done

```

The smoke test is successful if:

1. The container runs all four benchmarks without crashing.
2. The four log files are copied back to the host.
3. Each log contains the executed command line and the corresponding solver/LLM output.

To leave the container after the run:

```
exit
```

1.5 Reproducibility Runs: Complete Evaluation Instructions

To run experiments over all the benchmarks use the following. You need to set the appropriate model as required.

1.5.1 Model list for evaluation

Some model configurations have been presented below, use any one of them for the test. `BASE_URL_156` ollama instance is used with `--use156` and `BASE_URL_160` ollama instance is used with `--use160` flag. Change in the `parallel.sh` files accordingly.

```
model="--use156 --model gpt-oss:20b"
model="--use160 --model gpt-oss:20b"
model="--usebedrock --model us.meta.llama4-maverick-17b-instruct-v1:0"
model="--usebedrock --model qwen.qwen3-coder-30b-a3b-v1:0"
model="--usebedrock --model us.anthropic.claude-opus-4-6-v1"
model="--usebedrock --model openai.gpt-oss-120b-1:0"
model="--usegpt --model gpt-5.1"
```

1.5.2 Directory Naming Convention

The scripts expect the experimental log files to be organized in specific directories. The directory names are used throughout the workflow and should **not** be changed unless the scripts are modified accordingly. You need to move the `logFilesAll` assets accordingly to the `/export` folder as per below naming convention.

The expected directory structure is:

Model / Configuration	Directory Name	Model Name
GPT-OSS-20B (Local LLM)	<code>gpt20b</code>	<code>gpt-oss:20b</code>
GPT-OSS-120B (Amazon Bedrock)	<code>gpt120b</code>	<code>openai.gpt-oss-120b-1:0</code>
Claude (Amazon Bedrock)	<code>claude</code>	<code>us.anthropic.claude-opus-4-6-v1</code>
Qwen-30B (Amazon Bedrock)	<code>qwen30b</code>	<code>qwen.qwen3-coder-30b-a3b-v1:0</code>
Maverick (Amazon Bedrock)	<code>maverick</code>	<code>us.meta.llama4-maverick-17b-instruct-v1:0</code>
NoLemma Ablation	<code>NoLemma</code>	
NoGamma Ablation	<code>NoGamma</code>	

1.5.3 Additional notes on folder structure to save logs

- Log files generated using the **local GPT-OSS-20B model** should be stored in the `gpt20b` directory.
- Log files generated using **Amazon Bedrock models** should be stored in their respective directories (e.g., `claude`, `maverick`, etc.).
- Each directory should contain log files generated by **only one model/configuration**.
- The directory names should exactly match those listed above, as the scripts expect these names.

1.5.4 CLI flags supported by the tool

Bytecode compiled 8900 files in 136ms

```
usage: main.py [-h] -i INPUTFILE [-bench BENCHMARKNAME]
               [-t ITERATIONS] [-v] [-dbg] [-u1] [-u2]
               [-C] [-B] [-W] -M MODEL
               [-V FUZZTIME] [-Z Z3TIME] [-S SHAREDLIB]
               [-a MINLEMMA] [-b MAXLEMMA] [-g ADDGAMMA] [-l ADDLEMMA]
```

A script that accepts arguments.

options:

```
-h, --help            show this help message and exit
-i, --inputFile INPUTFILE
                       relative path to json file containing
                       information about the benchmarks
-bench, --benchName BENCHMARKNAME
                       Name of the benchmark file.
-t, --iterations ITERATIONS
                       Number of iterations made to LLM.
-v, --verbose          Enable verbose output.
-dbg, --debug          Enable Debug wrapped logs or information.
-u1, --usel56          Use the 156 local server.
-u2, --usel60          Use the 160 local server.
-C, --usegpt           Use ChatGPT models from OPENAI.
                       Set the .env file with CHAT_OPENAI_API_KEY
-B, --usebedrock       Use LLM models from AWS Bedrock.
                       Set the .env file with
                       AWS_BEARER_TOKEN_BEDROCK and AWS_REGION
-W, --stop             Stop running the framework.
-M, --model MODEL      Specify model name to use.
-V, --fuzztime FUZZTIME
                       Specify Fuzzer timeout in seconds. (Default 20s)
-Z, --z3time Z3TIME    Specify Z3 timeout in seconds. (Default 20s)
-S, --sharedLib SHAREDLIB
                       Link to the shared library. (Default "None")
-a, --minLemma MINLEMMA
                       Minimum number of lemmas to ask
                       LLM for at a time. (Default 1)
-b, --maxLemma MAXLEMMA
                       Maximum number of lemmas to ask
                       LLM for at a time (Default 5)
-g, --addGamma ADDGAMMA
                       If set zero then does not add input
                       output of functions to the z3 solver
-l, --addLemma ADDLEMMA
                       If set zero then does not add
                       LLM generated lemmas to the z3 solver
```

1.5.5 Running all 193 benchmarks (baseline)

The complete list of benchmarks are spread over three files that control the configuration. It takes 25 hours to run over all 193 benchmarks for a single model.

```
./runUnsat.sh benchLocationsNew
mv logFilesAll/* /export/<MODEL>

./runUnsat.sh benchLocationsNew1
mv logFilesAll/* /export/<MODEL>

./runUnsat.sh benchLocationsUnsat
mv logFilesAll/* /export/<MODEL>
```

MODEL is one of the folder model names as described in the table above (see [Directory Naming Convention](#) section)

1.5.6 For all 193 benchmarks with No Gamma experiment (NoGamma Ablation)

The complete list of benchmarks are spread over three files that control the configuration. You need to set the model in [parallelNoGamma.sh](#) (see the comment above for model list). It takes 25 hours to run over all 193 benchmarks for a single model. Default model used is **gpt-oss:20b** via Ollama.

```
# Default model used, change it in parallelNoGamma.sh
model="--model gpt-oss:20b --use156"
```

Running over all 193 benchmarks

```
./runUnsatNoGamma.sh benchLocationsNew
mv logFilesAll/* /export/NoGamma

./runUnsatNoGamma.sh benchLocationsNew1
mv logFilesAll/* /export/NoGamma

./runUnsatNoGamma.sh benchLocationsUnsat
mv logFilesAll/* /export/NoGamma
```

1.5.7 For all 193 benchmarks with No Lemma experiment (NoLemma Ablation)

The complete list of benchmarks are spread over three files that control the configuration. You need to set the model in [parallelNoLemma.sh](#) (see the comment above for model list). It takes 25 hours to run over all 193 benchmarks for a single model. Default model used is **gpt-oss:20b** via Ollama.

```
# Default model used, change it in parallelNoLemma.sh
model="--model gpt-oss:20b --use156"
```

Running over all 193 benchmarks

```
./runUnsatNoLemma.sh benchLocationsNew
mv logFilesAll/* /export/NoLemma

./runUnsatNoLemma.sh benchLocationsNew1
mv logFilesAll/* /export/NoLemma

./runUnsatNoLemma.sh benchLocationsUnsat
mv logFilesAll/* /export/NoLemma
```

1.6 Reusability Notes for benchmarks

1. The artifact is packaged as a Docker image so reviewers do not need to reproduce the Python or system dependency setup manually.

2. The image contains the repository, dependencies, and benchmarks required for execution.
3. The smoke test is intended to finish quickly (30 mins) and provide a basic end-to-end validation path.
4. Add a `benchmark` with appropriate JSON file and add the entry in `benchSmokeTest` to run. A sample benchmark structure is `./benchmarks/unsat_examples/example_1`
 - You need to add a `JSON file` with natural description and benchmark details.
 - `C code file` to expose the `closed-box` function as a `functional` module (run to get input/output).
 - `DESC` and compile commands for `C file` in `Makefile`.

You can modify `parallel.sh`, `runUnsat.sh`, or the benchmark manifests inside the docker container while running the benchmarks.

1.7 Troubleshooting

1. Please provide the API Key in `CHAT_OPENAI_API_KEY`. Add `CHAT_OPENAI_API_KEY` to `.env` and restart the container.
2. Please provide the AWS Bedrock token in `AWS_BEARER_TOKEN_BEDROCK`. Ensure `AWS_BEARER_TOKEN_BEDROCK` is present and non-empty in `.env`.
3. Bedrock authentication failures such as `AccessDeniedException`. Verify `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, optional `AWS_SESSION_TOKEN`, `AWS_REGION`, and model access permissions.
4. The container starts but cannot find output on the host. Make sure `docker run` included `-v "$PWD:/export"` and that `mv logFilesAll/* /export/` was executed inside the container.